GPU Architecture and Programming

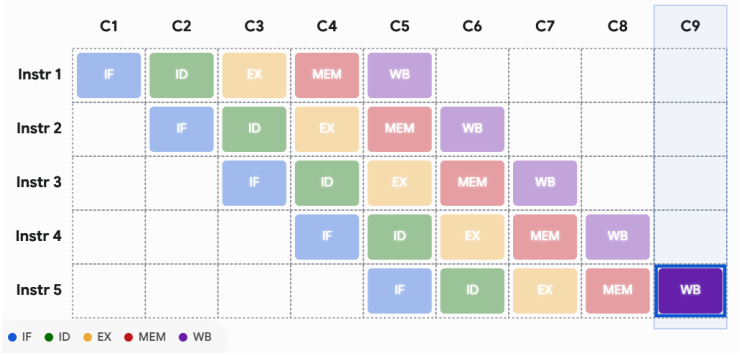
The operation of a processor is characterised by fetch → decode → execute cycle

RISC/CISC – Reduced / Complex Instruction Set Computing. Approach of CISC is to complete a task with as few instruction as possible.

CISC features	RISC features
• Multicycle instructions, HW intensive design • Efficient RAM usage • Instruction – complex and variable length, lots of them • Compound addressing modes • Micro-code support	• Single-cycle instructions, SW intensive design • Heavy RAM usage, Large Register file • Small no. Of simple fixed length instruction • Less no. of addressing modes

5 stage pipeline to execute code:

1. Instruction Fetch(IF): Fetches instruction from memory address stored at the Ptogram Counter (PC), loads into Instruction Register (IR)
2. Instruction Decode (ID): CPU's control unit reads binary code sitting in IR and figures out what operation is requested
3. Execute (EX): Actual computation inside ALU
4. Memory(MEM): If instruction requires reading from or writing to RAM, it happens here. If a simple math calculation, the processor essentially skips.
5. Writeback(WB): The final result if execution is written back into CPU's internal register



When this works perfectly, an instruction completes every single clock cycle, maximizing throughput. Situations that prevent the next instruction from executing in its designated clock cycle. These situations are called **hazards**.

Structural Hazards – The hardware does not support the combination of instructions that are scheduled to execute in the same clock cycle. It is fundamentally a resource conflict. Example: Instruction-1 is currently in the MEM stage, reading data from memory. At that exact same clock cycle, Instruction-4 is in the IF stage, trying to fetch the next instruction from memory. If the hardware can only handle one memory access at a time, a structural hazard occurs

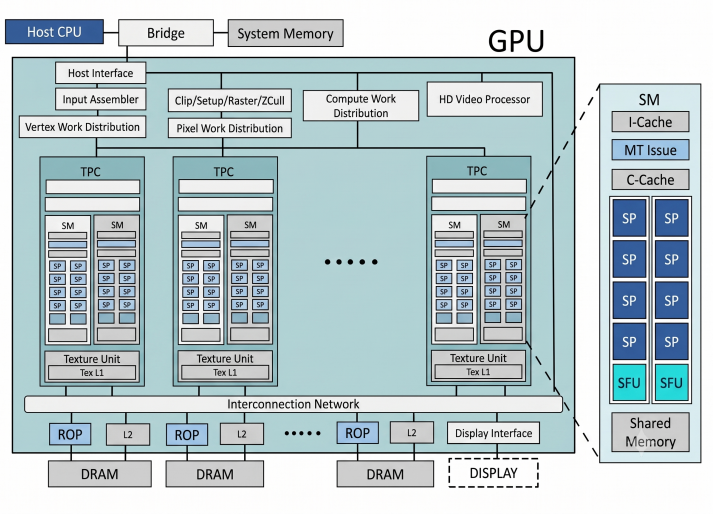
Data Hazards – When an instruction depends on the result of a previous instruction, but that result has not yet been saved or made available by the pipeline. Example I1: ADD R1, R2, R3 (Adds R2 and R3, saves to R1); I2: SUB R4, R1, R5 (Subtracts R5 from R1) I1 calculates the new value for R1 in the EX stage but doesn't write it to the register file until the WB stage (cycle 5). However, I2 needs to read R1 during its ID stage (cycle 3). I2 is trying to read the data before I1 has written it.

Control Hazards – Arise from instructions that change the flow of a program, such as branches, jumps, or function calls. Example CPU fetches a branch instruction (like an if statement), it doesn't immediately know whether the branch will be taken or not, nor does it know the target address. This condition usually isn't resolved until the EX or MEM stage. Meanwhile, the pipeline has already blindly fetched the next several instructions in the sequence. If the branch is taken, all those blindly fetched instructions are the wrong ones.

Cache mapping is the rule or algorithm that a computer uses to determine where a specific block of main memory (RAM) should be stored in the much smaller cache memory. When the CPU requests data, it generates a main memory address. The cache mapping strategy dictates how that address is

mathematically sliced up (into a Tag, Index, and Offset) to quickly check if the data is already in the cache (a cache hit) or if it needs to be fetched from RAM (a cache miss).

GPU Architecture



System and Host Interface The GPU connects to the Host CPU and System Memory via a Bridge. The Host Interface manages this communication, feeding data and commands into the GPU pipeline.
Work Distribution and Frontend The Input Assembler prepares raw geometry data. Various units, such as Vertex Work Distribution, Pixel Work Distribution, and Compute Work Distribution, manage how tasks are spread across the available hardware resources.
Processing Blocks (TPCs) The core of the GPU consists of several scalable Texture Processing Clusters (TPCs). Each TPC contains smaller units called SMs (Streaming Multiprocessors) and dedicated Texture Units for graphical operations.
SM Microarchitecture The detailed breakout on the right focuses on a single Streaming Multiprocessor (SM), revealing its internal structure:

- Caches (I-Cache, C-Cache): Store instruction and constant data.
- MT Issue: Manages multithreaded scheduling and issue of operations.
- Cores: Multiple execution units, including SP (Streaming Processors) for general arithmetic/pixel work and SFU (Special Function Units) for complex transcendental math (like sine or logarithm).
- Shared Memory: Fast, local memory shared among the threads running on that SM.

Memory Hierarchy and Output An Interconnection Network handles communication between the processing units and the memory system. Data flows to ROP (Raster Operation) units and L2

Caches, which interface directly with dedicated DRAM blocks. Finally, the Display Interface routes finished pixels to the physical DISPLAY.